DIGITAL ELECTRONICS

ASSIGNMENT 1 TO 9

Name: _____

Roll No.: _____

Section: _____

Submitted To: _____

GitHub Repository: _____

Contents

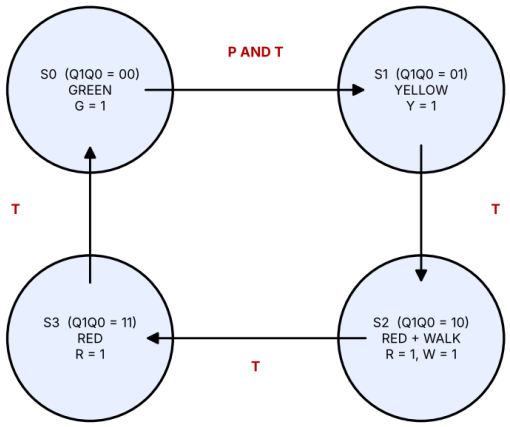
ASSIGNMENT	TITLE	PAGE
1	FSM Design for a Traffic Light Controller with Pedestrian Request	3
2	Sequential Circuits in Real Systems - Notes	5
3	4-bit ALU, Program Counter and Code-to-Hardware Mapping	8
4	GenAI-Assisted Code-to-Hardware Mapping	10
5	Natural Language to Logic Circuit Generator	12
6	AI-Assisted Boolean Expression Simplifier	14
7	AI-Based Fault Detection in a Digital Full Adder	16
8	Automatic Parking-Barrier Controller (Combinational + Sequential)	18
9	Digital Clock FSM Design with GenAI (24-Hour)	21

FSM Design for a Traffic Light Controller with Pedestrian Request

1. Prompt. GenAI was asked to act as a digital logic designer and produce, for a 3-light traffic controller with a pedestrian request input P and timer-expired input T: (a) a state diagram, (b) a present/next-state table, (c) output equations for Green, Yellow, Red and Walk, (d) next-state equations for two D flip-flops, and (e) a Logisim simulation plan with a test sequence.

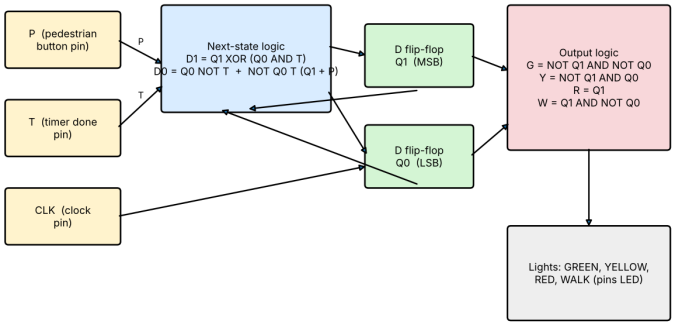
2. Prompt response. A four-state Moore machine (Q1Q0): S0 Green, S1 Yellow, S2 Red+Walk, S3 Red. S0 leaves only when P AND T; S1, S2, S3 advance when T = 1.

Figure 1. State diagram of the traffic light FSM (Moore machine)



Each state holds while its condition is false. S0 leaves only when P = 1 and T = 1. S1, S2, S3 advance one step whenever T = 1 (the timer has expired).

Figure 2. Logisim block structure: D flip-flops, next-state logic, output decoder



State table

Present	Q1Q0	P	T	Next	Next Q1Q0	G	Y	R	W
S0 GREEN	00	0	0	S0 GREEN	00	1	0	0	0
S0 GREEN	00	0	1	S0 GREEN	00	1	0	0	0
S0 GREEN	00	1	0	S0 GREEN	00	1	0	0	0
S0 GREEN	00	1	1	S1 YELLOW	01	1	0	0	0
S1 YELLOW	01	0	0	S1 YELLOW	01	0	1	0	0
S1 YELLOW	01	0	1	S2 RED+WALK	10	0	1	0	0
S1 YELLOW	01	1	0	S1 YELLOW	01	0	1	0	0
S1 YELLOW	01	1	1	S2 RED+WALK	10	0	1	0	0
S2 RED+WALK	10	0	0	S2 RED+WALK	10	0	0	1	1
S2 RED+WALK	10	0	1	S3 RED	11	0	0	1	1
S2 RED+WALK	10	1	0	S2 RED+WALK	10	0	0	1	1
S2 RED+WALK	10	1	1	S3 RED	11	0	0	1	1
S3 RED	11	0	0	S3 RED	11	0	0	1	0
S3 RED	11	0	1	S0 GREEN	00	0	0	1	0
S3 RED	11	1	0	S3 RED	11	0	0	1	0
S3 RED	11	1	1	S0 GREEN	00	0	0	1	0

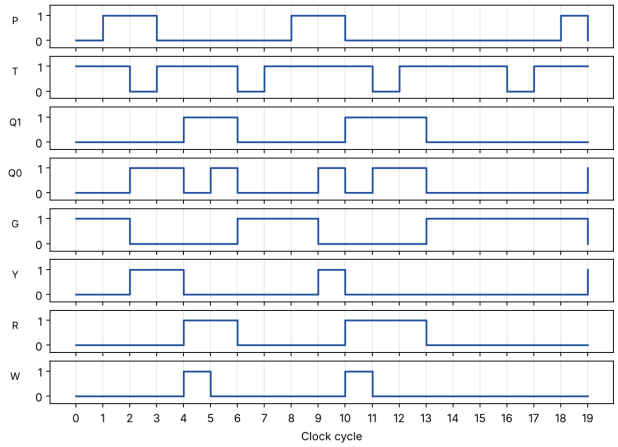
Output equations
G = NOT Q1 AND NOT Q0
Y = NOT Q1 AND Q0
R = Q1
W = Q1 AND NOT Q0

Next-state equations (D flip-flops)
D1 = Q1 XOR (Q0 AND T)
D0 = (Q0 AND NOT T) OR (NOT Q0 AND T AND (Q1 OR P))
Checked against the state table for all 16 input cases: all match.

3. Logisim implementation. Two D flip-flops (Memory library) hold Q1 and Q0, clocked by a Clock component. Build D1 and D0 from AND, OR, NOT and XOR gates using pins P and T; build G, Y, R, W from the output equations and drive LED pins. Initial state 0 (Green). Figure 2 shows the structure. Insert your Logisim circuit and simulation screenshots here, since they must come from your own Logisim run.

4. Result. The sequence below was applied for 20 clock cycles (P pulses at cycles 1, 2, 8, 9, 18). The controller stays Green until a request is accepted while T = 1, then runs Yellow, Red+Walk, Red and returns to Green. Figure 3 shows the waveforms.

Figure 3. Simulated waveforms of the traffic light FSM



Cyc	P	T	Q1Q0	State	G	Y	R	W
0	0	1	00	S0 GREEN	1	0	0	0
1	1	1	00	S0 GREEN	1	0	0	0
2	1	0	01	S1 YELLOW	0	1	0	0
3	0	1	01	S1 YELLOW	0	1	0	0
4	0	1	10	S2 RED+WALK	0	0	1	1
5	0	1	11	S3 RED	0	0	1	0
6	0	0	00	S0 GREEN	1	0	0	0
7	0	1	00	S0 GREEN	1	0	0	0
8	1	1	00	S0 GREEN	1	0	0	0
9	1	1	01	S1 YELLOW	0	1	0	0
10	0	1	10	S2 RED+WALK	0	0	1	1
11	0	0	11	S3 RED	0	0	1	0
12	0	1	11	S3 RED	0	0	1	0
13	0	1	00	S0 GREEN	1	0	0	0
14	0	1	00	S0 GREEN	1	0	0	0
15	0	1	00	S0 GREEN	1	0	0	0
16	0	0	00	S0 GREEN	1	0	0	0
17	0	1	00	S0 GREEN	1	0	0	0
18	1	1	00	S0 GREEN	1	0	0	0
19	0	1	01	S1 YELLOW	0	1	0	0

5. Conclusion. The Moore FSM with two D flip-flops meets the requirements: Green holds until a valid pedestrian request, the sequence Yellow, Red+Walk, Red and back to Green is followed, and no unsafe light combination appears. GenAI output was verified against the state table before implementation.

Block 1 - Flip-Flop (Exp. 17): 1-bit register cell & keypad debounce

CO3 (primary) | CO1 (reinforced) | Real systems: CPU register bit; matrix-keypad / push-button debounce

1. Prompt

Identify a real digital system in which the edge-triggered D flip-flop built in the lab is the core storage element. Explain how one flip-flop acts as a 1-bit memory cell and how n of them form a CPU register, then explain how a flip-flop-based latch removes contact bounce from a keypad key so that the processor sees exactly one clean edge. Name each real system, state the block's role, justify why a flip-flop (and not a plain gate) is required, and support the claim with a small simulation.

2. Prompt Response

- **System 1 - CPU register (e.g. accumulator / program-counter bit).** A register of n bits is n D flip-flops sharing one clock. At every rising edge each flip-flop copies its D input to Q and then holds it, ignoring changes on D until the next edge. That 'sample on the edge, hold between edges' behaviour is exactly what makes it 1 bit of memory; a combinational gate has no such memory.
- **System 2 - Keypad / push-button debounce.** A mechanical key bounces for ~1-20 ms, producing many 0-1 transitions. An SPDT key driving a cross-coupled NAND SR latch (the basic cell inside every flip-flop) sets Q on the first contact and cannot be reset until the opposite contact is touched, so later bounces are ignored. The CPU reads one clean level/edge.
- **Why it fits:** bistable feedback (Q holds its state) is the common principle in both systems; the D-FF adds clock-edge control so the register bits change in step with the CPU.

3. Logisim Implementation

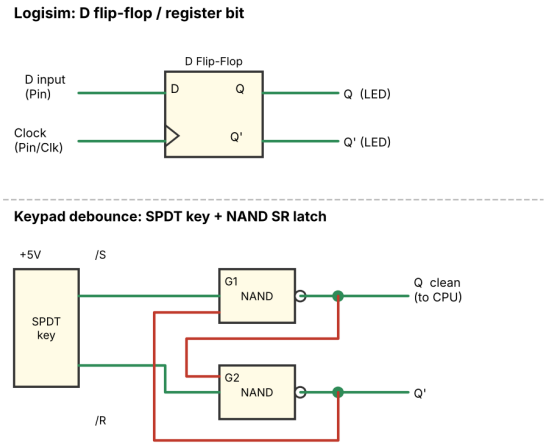


Fig 1: Logisim circuit - D flip-flop with Pin/Clock/LED, and SPDT + NAND SR-latch debouncer

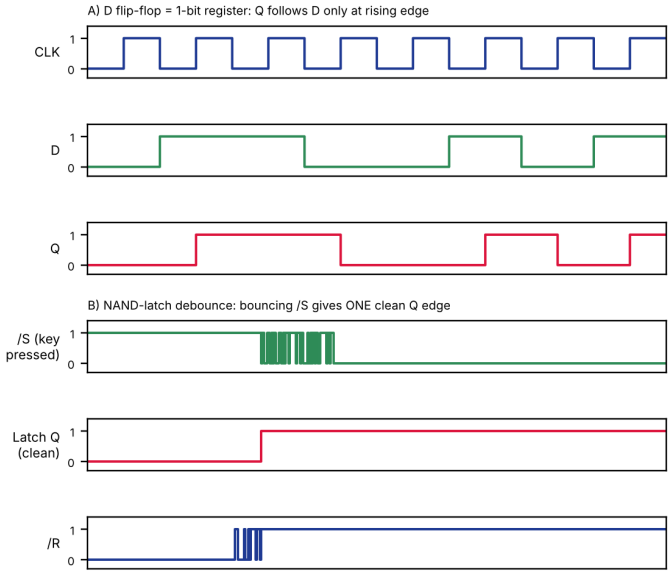


Fig 2: Simulation output - (A) register bit, (B) debounce

4. Result

- (A) With D changing between clock edges, Q updated **only** at the rising CLK edges and held its value in between (Q sequence at successive edges: 0,1,1,0,0,1,0,1 for D = 0,1,1,0,0,1,0,1).
- (B) The bouncing key line /S produced **35 raw transitions**, while the latch output Q produced **exactly 1** clean transition (0 to 1).

5. Conclusion

The flip-flop of Exp. 17 is the 1-bit cell of every CPU register, and the same bistable feedback inside it debounces mechanical keys. The simulation confirms (i) edge-triggered capture and hold of D, and (ii) elimination of bounce (35 transitions reduced to 1). This links sequential-logic theory (CO3) to real hardware behaviour (CO1).

Note: schematics are redrawn in Logisim style; waveforms are from a Python behavioural simulation of the same logic.

Block 2 - Mod-N Counter (Exp. 18): Mod-60 in a digital clock

CO3 (primary) | CO1 (reinforced) | Real systems: digital-clock seconds/minutes counter; clock-generation frequency divider; traffic-light timer

1. Prompt

Build a Mod-60 counter from the Mod-N counters of the lab and connect it to a real timekeeping system. Show how a Mod-10 (units) and a Mod-6 (tens) counter are cascaded to count 00-59, how the roll-over from 59 to 00 produces a carry that clocks the minutes stage in a digital clock, how the same divide-by-N idea gives a 1 Hz tick from a crystal oscillator, and how it can time traffic-light phases. State the role of the block in each system and demonstrate the Mod-60 counter in simulation.

2. Prompt Response

- **System 1 - Digital clock.** A 32.768 kHz quartz crystal is divided by 2^{15} (15 flip-flops) to give exactly 1 Hz. The seconds counter is a Mod-10 stage (0-9) cascaded with a Mod-6 stage (0-5): 10 x 6 = 60 states. When it rolls 59 to 00 its carry output is a 1-per-minute pulse that clocks the minutes counter (also Mod-60) and then the hours counter (Mod-24/12).
- **System 2 - Frequency divider in clock generation.** A Mod-N counter outputs one pulse per N inputs, so $f_{out} = f_{in}/N$. Chains of counters derive baud, tick and refresh clocks from one crystal.
- **System 3 - Traffic-light timer.** A counter driven by a 1 Hz tick is compared/reset at preset counts (e.g. green 0-29, amber 30-34, red 35-59) so one Mod-60 cycle is one signal cycle; a decoder on the count drives the lamps.

3. Logisim Implementation

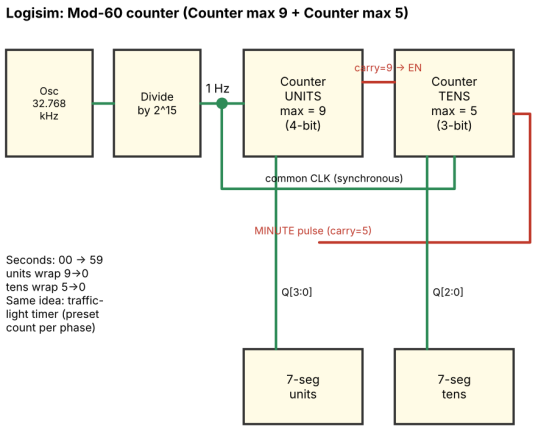


Fig 3: Logisim circuit - Counter(max 9) + Counter(max 5), shared clock, carry to EN, 7-segment outputs

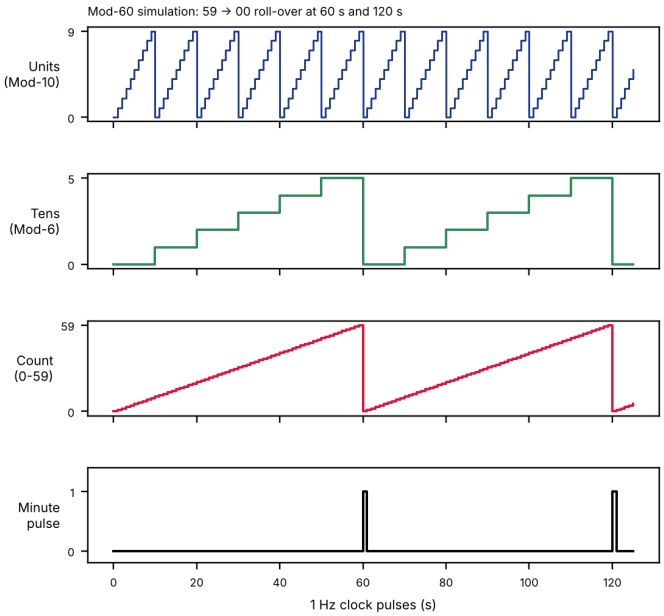


Fig 4: Simulation of 125 clock pulses

4. Result

- Over 125 pulses the combined count (10 x tens + units) equalled (pulse number mod 60) at **every** step (verified programmatically); units wrapped 9 to 0 every 10 pulses and tens wrapped 5 to 0 every 60.
- The roll-over 59 to 00 occurred at pulses **60 and 120**, each producing one minute-carry pulse.

5. Conclusion

Cascading a Mod-10 and a Mod-6 counter gives a Mod-60 counter (10 x 6 = 60) with a carry that is exactly the 'minute' tick of a digital clock. The same divide-by-N principle realises the 1 Hz divider and phase timing of a traffic light, so the Exp. 18 counter maps directly onto real timekeeping and control systems.

Block 3 - Shift Register (Exp. 19): SIPO/PISO in UART/SPI & LED displays

CO3 (primary) | CO1 (reinforced) | Real systems: UART / SPI serial link; LED-display drivers (e.g. 74HC595-type)

1. Prompt

Relate the serial-in parallel-out (SIPO) and parallel-in serial-out (PISO) shift registers built in the lab to real systems. Explain serial-to-parallel conversion at the receiver and parallel-to-serial conversion at the transmitter of a UART or SPI link, and explain how a SIPO register lets a microcontroller drive many LEDs of a display with few pins. Name each system, describe the role of each register type, and demonstrate SIPO and PISO behaviour in a simulation.

2. Prompt Response

- **System 1 - UART / SPI.** A transmitter holds a byte in parallel; a PISO register loads it and shifts it out one bit per clock on a single wire (TX / MOSI). The receiver's SIPO register shifts the incoming bits in and, after 8 clocks, presents the whole byte on 8 parallel lines for the CPU. Cascading two 4-bit lab registers gives the 8-bit version.
- **System 2 - LED display / multiplexing.** To drive 8 or more segments or LEDs, the microcontroller sends the pattern serially (data + clock) to a SIPO register (e.g. 74HC595) and a latch pulse then updates all outputs at once. Repeating this for each digit rapidly (multiplexing) shows a multi-digit display using only 3-4 MCU pins.
- **Why it fits:** the shift register is the hardware trade between pins and time: n wires become 1 wire sent over n clock cycles, and back again.

3. Logisim Implementation

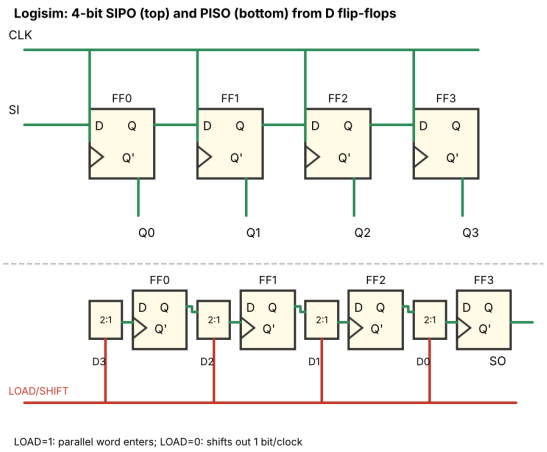


Fig 5: Logisim circuit - 4 D flip-flops as SIPO (top) and with 2:1 load muxes as PISO (bottom)

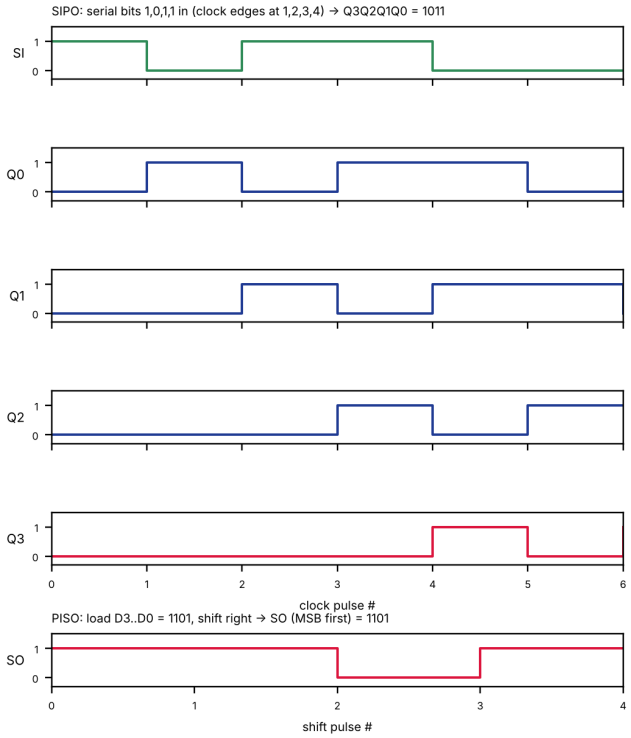


Fig 6: Simulation - SIPO timing and PISO serial output

4. Result

- SIPO: serial bits 1,0,1,1 applied on 4 successive clocks gave parallel output Q3Q2Q1Q0 = **1011** (first bit has travelled to Q3).
- PISO: parallel word 1101 loaded, then shifted right, produced the serial stream **1, 1, 0, 1** (MSB first), one bit per clock.

5. Conclusion

The Exp. 19 shift registers perform the two conversions that every serial link needs - parallel to serial at the sender and serial to parallel at the receiver - and the same SIPO structure expands a few MCU pins into many LED outputs. The simulation shows the exact bit order and the 4-clock delay, confirming the data-width versus time trade-off.

Note: schematics are redrawn in Logisim style; waveforms are from a Python behavioural simulation of the same logic.

4-bit ALU, Program Counter and Code-to-Hardware Mapping

Logisim design & simulation report | CO4 (primary), CO1 (reinforced)

1. Prompt

Prompt used: "Design a 4-bit ALU made of a 4-bit adder/subtractor in which a 2:1 mode-select MUX (select M) chooses B or NOT B for every bit, with M also driving the carry-in, so that M=0 gives A+B and M=1 gives A-B. Also design a simple 4-bit program counter (up-counter) from D flip-flops with enable and reset inputs. Show how both are simulated in Logisim, then take a line of code such as *c = a + b;* or a loop *for(i=0;i<8;i++)* and identify (a) which block does the arithmetic (ALU), (b) which blocks hold operand values (registers / flip-flops) and (c) which block sequences instructions (program counter incrementing each iteration). Present the mapping as a labelled diagram with a short write-up, the simulation results and a conclusion."

2. Prompt Response (design)

ALU. Each bit of B passes through a 2:1 MUX whose inputs are B (M=0) and NOT B (M=1). The MUX outputs feed a chain of four full adders (ripple carry) with A. Carry-in of FA0 = M, so for M=1 the circuit forms A + (~B) + 1 = A - B (two's complement). Outputs: S3..S0 and Cout (for subtraction Cout=1 means no borrow, Cout=0 means A<B). **Program counter.** A 4-bit register of D flip-flops holds PC. A +1 adder produces PC+1; a MUX controlled by EN selects PC+1 (EN=1) or the old PC (EN=0) as the D input; RST=1 clears PC to 0000; the value advances on each clock rising edge.

3. Logisim Implementation

Logisim parts: Input pins A[4], B[4], M, EN, RST; Multiplexer (1 bit, 2 inputs) x4 + NOT gates; Adder (4-bit, Cin/Cout enabled) or 4 full adders; Register/D flip-flop x4 (4-bit, with enable and async reset); Adder 4-bit (+1 constant); Clock; Output pins/hex displays. Schematics below show the wiring used.

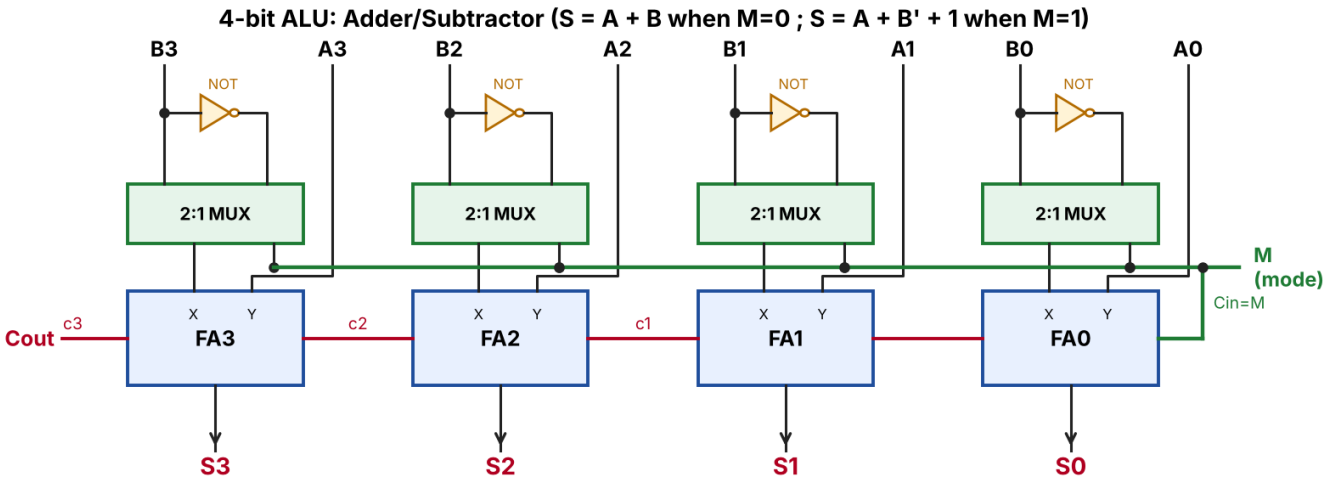


Fig. 1: 4-bit ALU (adder/subtractor with mode-select MUX)

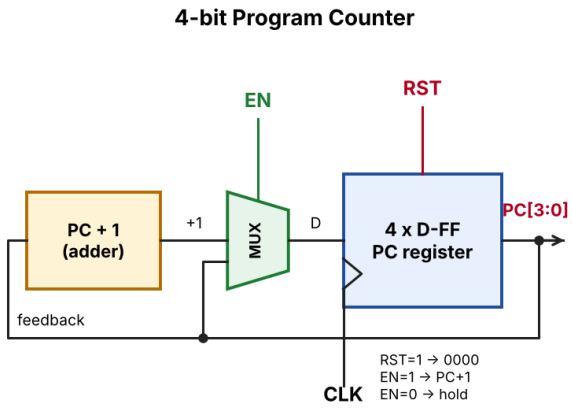


Fig. 2: 4-bit program counter

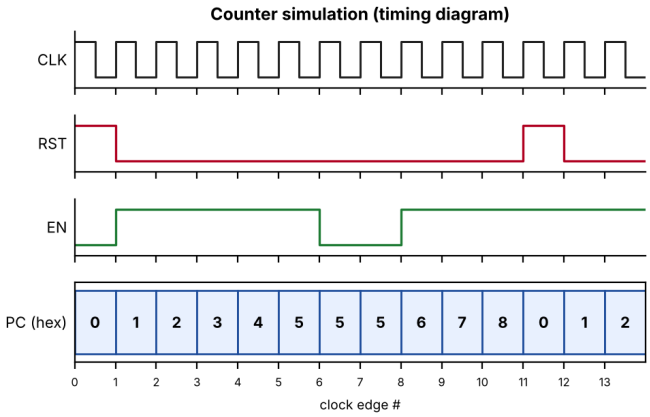


Fig. 3: counter timing diagram

4. Result

(a) ALU truth-table check (verified with a Python model of the same gate-level logic):

A	B	M	Operation	S (bin)	S (dec)	Cout
0101	0011	0	A+B	1000	8	0
1001	0100	1	A-B	0101	5	1

A	B	M	Operation	S (bin)	S (dec)	Cout
0011	0101	1	A-B	1110	14	0
1001	0111	0	A+B	0000	0	1
1111	0001	0	A+B	0000	0	1
0110	0110	1	A-B	0000	0	1

3-5 = 1110 (-2 in two's complement; Cout=0 = borrow). 9+7 and 15+1 overflow to 0000 with Cout=1. 6-6 = 0000 with Cout=1.

(b) Counter (Fig. 3): RST=1 gives 0; EN=1 counts 1,2,3,4,5; EN=0 holds 5; counting resumes 6,7,8; RST has priority and returns to 0; counting restarts 1,2. After 15 the counter wraps to 0.

(c) Code mapping (Fig. 4) and loop trace for *for(i=0;i<8;i++)*:

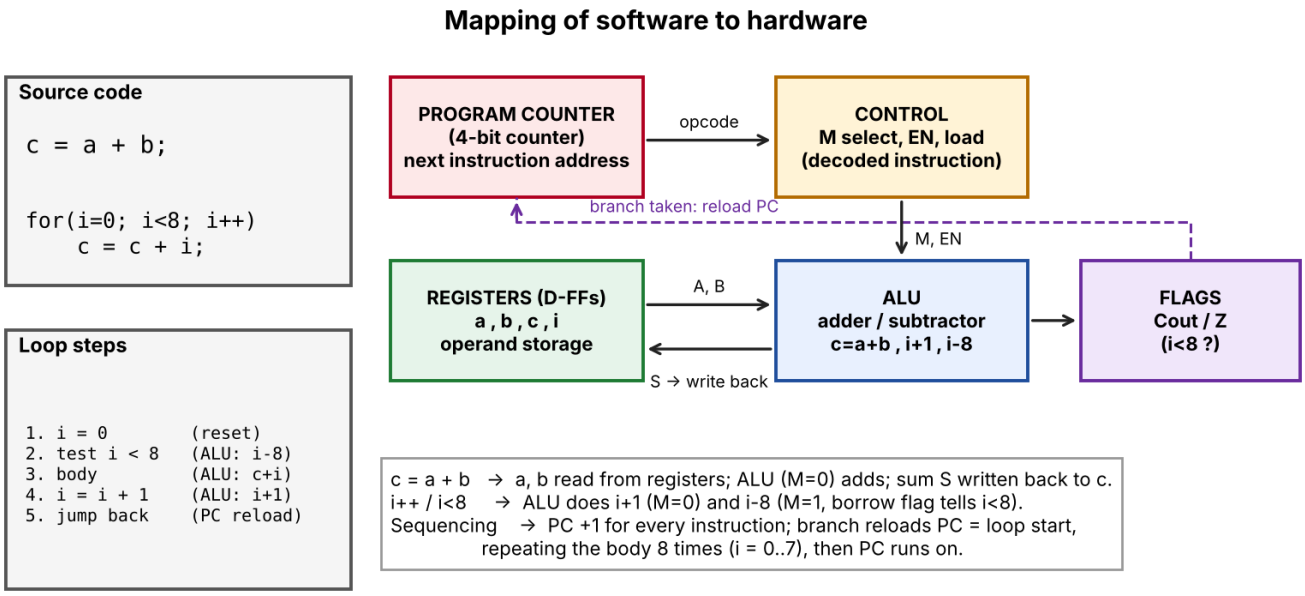


Fig. 4: Labelled mapping of code to ALU, registers and program counter

Pass	i (reg)	ALU: i-8 (M=1)	Cout (no borrow)	i<8?	ALU: i+1 (M=0)	PC action
1	0	1000	0	Yes	1	jump to loop start
2	1	1001	0	Yes	2	jump to loop start
3	2	1010	0	Yes	3	jump to loop start
4	3	1011	0	Yes	4	jump to loop start
5	4	1100	0	Yes	5	jump to loop start
6	5	1101	0	Yes	6	jump to loop start
7	6	1110	0	Yes	7	jump to loop start
8	7	1111	0	Yes	8	jump to loop start
9	8	0000	1	No	-	exit loop (PC+1)

Code element	Hardware block
c = a + b;	a, b: registers (D flip-flops) -> ALU (M=0) -> S written back to register c
i = 0	Register i cleared (RST)
i < 8	ALU subtract (M=1): i - 8, borrow flag (Cout=0) means true
i++	ALU add (M=0) with B=1, result stored in i
Next instruction / loop jump	Program counter: +1 each instruction, reloaded with loop start on branch

5. Conclusion

The 4-bit ALU performed addition (M=0) and two's-complement subtraction (M=1) correctly using a MUX-selected B input and carry-in, and the enable/reset program counter counted, held, reset and wrapped as specified. Mapping the code shows that arithmetic (c=a+b, i+1, i-8) is done by the **ALU**, operands a, b, c, i are stored in **registers (flip-flops)**, and instruction sequencing and the 8 loop iterations are controlled by the **program counter** with a branch on the ALU flag. This links program statements directly to digital hardware blocks (CO4) built from basic gates and flip-flops (CO1).

GenAI-Assisted Code-to-Hardware Mapping: if (a > b) c = a; else c = b;

Course outcomes: CO4 (primary), CO2 (AI-assisted design methodology)

1. Prompt

Act as a digital-logic design tutor. Take the software statement `if (a > b) c = a; else c = b;` for 4-bit unsigned inputs a and b, and map it onto hardware. Decide which of these building blocks are needed: comparator, multiplexer, adder, counter and register, and justify each choice or rejection. Then give the block-level structure, the gate-level design of the comparator, the multiplexer select wiring, and a check that your design's output matches the software behaviour for every input pair.

2. Prompt Response

Condensed response. Replace with the full output from your GenAI tool before submission.

Required blocks: A 4-bit comparator produces sel = (a > b). A 4-bit 2:1 multiplexer outputs a when sel = 1 and b when sel = 0. An adder is not needed because there is no arithmetic. A counter is not needed because there is no loop or state. A register is optional, and only needed if c must hold its value between clock edges. The circuit is purely combinational.

Block structure: a, b feed the comparator; its output sel drives the select input of four parallel 2:1 muxes; the muxes take a on I1 and b on I0.

Comparator gates (MSB to LSB): for each bit i, gt_i = a_i · NOT b_i and eq_i = XNOR(a_i, b_i). Then a > b = gt3 + eq3·(gt2 + eq2·(gt1 + eq1·gt0)). **Mux bit:** c_i = (sel · a_i) + (NOT sel · b_i).

Tie case: when a = b, both branches give the same value, so the tie does not change the output.

3. Logisim Implementation

Build in Logisim Evolution: add 4-bit input pins A and B and a 4-bit output pin C. Place the built-in *Comparator* (Arithmetic folder) set to unsigned, and connect its > output as the select line. Place a 4-bit *Multiplexer* (Plexers folder) with its input 1 wired to A and input 0 wired to B, and its output to C. The gate-level version uses NOT, AND, OR and XNOR gates following the equations in Section 2. Figure 1 shows the block-level design used for the build.

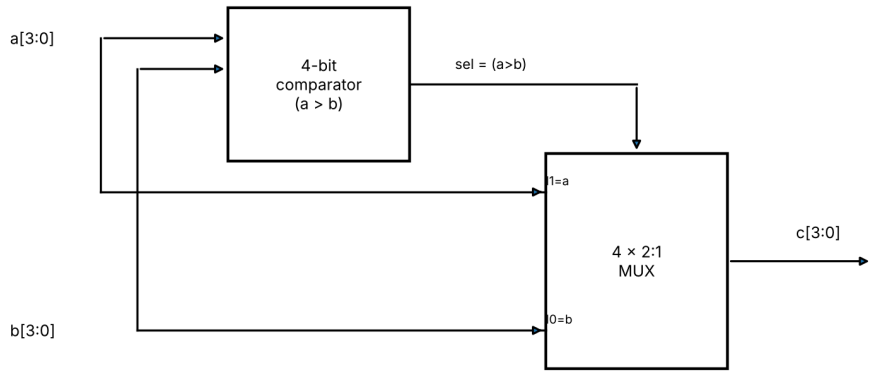


Figure 1. Block-level design: 4-bit comparator driving four 2:1 multiplexers.

4. Result

The design was checked against the software statement for all **256** input pairs (a, b in 0 to 15). Mismatches: **0**. Selected cases are below.

a	b	Expected sel	Expected c	Circuit c	Match
0000 (0)	0000 (0)	0	0000 (0)	0000 (0)	Yes
0101 (5)	1001 (9)	0	1001 (9)	1001 (9)	Yes
1100 (12)	0011 (3)	1	1100 (12)	1100 (12)	Yes

0111 (7)	0111 (7)	0	0111 (7)	0111 (7)	Yes
1111 (15)	0000 (0)	1	1111 (15)	1111 (15)	Yes
1001 (9)	1001 (9)	0	1001 (9)	1001 (9)	Yes

5. Conclusion

The if-else maps to one comparator and one multiplexer. Exhaustive checking confirmed the circuit's logic matches the code. The GenAI response was correct on the core structure but needed three corrections:

(i) It suggested an adder, reasoning that a > b could be found by subtraction. A comparator is simpler and adds no unused arithmetic logic. (ii) It did not explain how the comparator is built from gates, so the cascade equations were added and verified. (iii) It listed counter and register without noting that they are only needed for loops or stored values, which this snippet does not have. The tie case and unsigned interpretation were also made explicit.

Experiment repository: [paste your GitHub link here]. This document, the Python check script and the Logisim file (.circ) should be uploaded there.

Natural Language to Logic Circuit Generator

GenAI-assisted conversion of an English description into a Boolean expression and gate-level design, then simulation and verification

1. Prompt

> *Act as a digital logic design assistant. A home security panel works like this: the siren must sound when the panic button is pressed, or when the system is armed and either the door sensor or the window sensor detects an opening. Do the following in order: (1) assign a variable to every input and to the output; (2) translate the sentence into a Boolean expression; (3) build the truth table for all input combinations; (4) simplify the expression and name the Boolean laws used; (5) give the gate-level design with every connection; (6) give step-by-step instructions to build and simulate it in Logisim, plus the test cases needed to verify it.*

2. Prompt response

Variables: A = system armed, D = door sensor open, W = window sensor open, P = panic button (all 1 = active); ALARM = siren output.

Expression from the sentence: ALARM = P + A·(D + W). Expanded form: P + A·D + A·W (distributive law). The expanded form needs 4 two-input gates; the factored form needs only 3, so the factored form is used.

Gate-level design: OR1 takes D and W; the AND gate takes A and the OR1 output; OR2 takes the AND output and P and drives the ALARM output.

Logisim steps:

- Add 4 Input pins (A, D, W, P) and 1 Output pin (ALARM) from the Wiring library; label them.
- Place two OR gates and one AND gate from the Gates library (2 inputs each).
- Wire D, W to OR1; A and OR1 to AND; AND and P to OR2; OR2 to the output pin.
- Use Simulate > Poke to toggle inputs, or Project > Analyze Circuit to compare against the truth table.
- Test cases: all 16 combinations of A D W P (key cases: 0001, 1100, 1010, 0110, 1000).

3. Logisim implementation

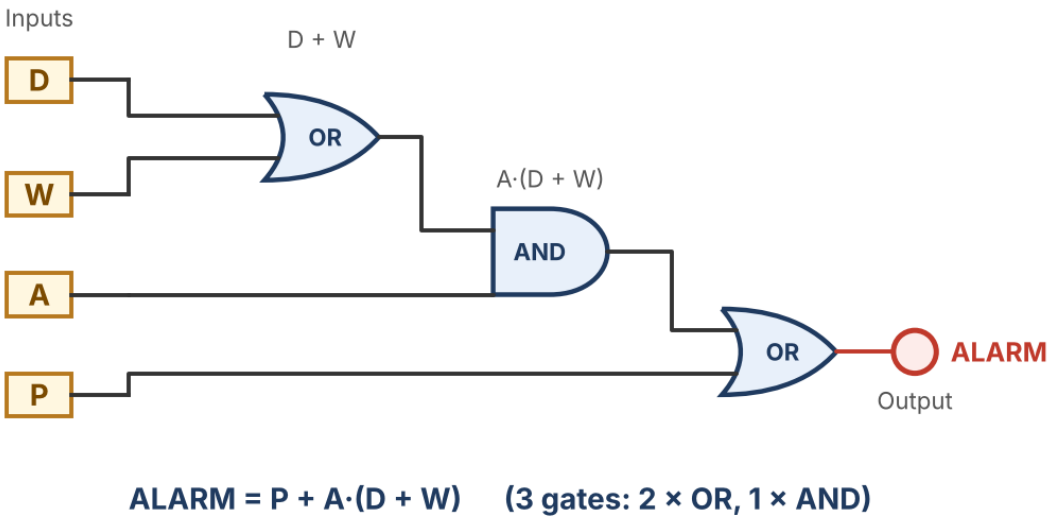


Fig. 1 – Gate-level circuit for ALARM = P + A·(D + W), as built in Logisim (inputs on the left, output on the right).

4. Result

Truth table: all 16 combinations

A	D	W	P	Expected	Circuit	Match
0	0	0	0	0	0	✓
0	0	0	1	1	1	✓
0	0	1	0	0	0	✓
0	0	1	1	1	1	✓
0	1	0	0	0	0	✓
0	1	0	1	1	1	✓
0	1	1	0	0	0	✓
0	1	1	1	1	1	✓
1	0	0	0	0	0	✓
1	0	0	1	1	1	✓
1	0	1	0	1	1	✓
1	0	1	1	1	1	✓
1	1	0	0	1	1	✓
1	1	0	1	1	1	✓
1	1	1	0	1	1	✓
1	1	1	1	1	1	✓

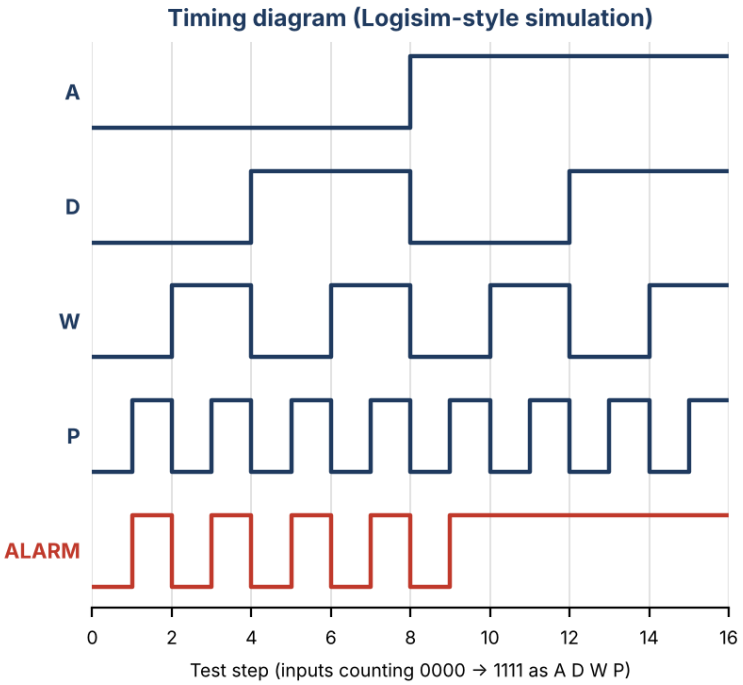


Fig. 2 – Simulation of all 16 input combinations: the circuit output matches the output expected from the English description in every row.

The circuit produced ALARM = 1 for **11 of 16** input combinations and 0 for the other 5. The alarm is on in all 8 cases with P = 1, and with P = 0 only when A = 1 and at least one of D, W = 1 (3 cases: 1100, 1010, 1110). With the system disarmed (A = 0) and no panic, door or window events never trigger the siren. All 16 rows matched (16/16 PASS). Longest path: D/W → OR1 → AND → OR2 (3 gate levels).

5. Conclusion

A plain-English requirement was converted by GenAI into a Boolean expression, a simplified 3-gate design and Logisim build steps. Simulation and exhaustive truth-table checking confirmed the design is correct. GenAI speeds up translation and simplification, but its output must still be verified by simulation, as done here, because wording ambiguities (for example, whether panic works when disarmed) can change the logic.

AI-Assisted Boolean Expression Simplifier

Function: F(A,B,C,D,E) = Σm(0,2,4,5,6,7,8,10,13,15,16,18,20,21,22,23,29,31) **Tools:** GenAI (tutor role), K-map, algebra, Logisim (Evolution / 2.7.1)

1. Prompt

Given to the GenAI as a digital-logic tutor: reduce the 5-variable canonical SOP F(A,B,C,D,E) above to minimum form, with **(a)** a 5-variable K-map (two 4×4 layers, A=0 and A=1) listing every group, prime implicant and essential prime implicant; **(b)** an algebraic cross-check; **(c)** the minimal SOP and minimal POS; **(d)** literal and gate counts before and after; **(e)** step-by-step Logisim build instructions (components, pin names, wiring) and a procedure to verify all 32 input combinations against the original truth table; **(f)** results as a truth table, K-map and circuit diagram, followed by a short conclusion.

Coverage of the project question: GenAI use (tutor prompt); 5 variables (within 4–6); SOP and POS (c); K-map and algebra (a, b); simulator implementation (e); circuit verification (e, f).

2. Prompt Response (GenAI output)

Minterms: F = 1 for 18 minterms and 0 for 14. **K-map groups** (rows BC, columns DE in Gray code; edges wrap):

Prime implicant	Cells covered	Essential?	Reason
C·E	m5, 7, 13, 15, 21, 23, 29, 31 (8)	Yes	Only cover of m13, 15, 29, 31
B'·E'	m0, 2, 4, 6, 16, 18, 20, 22 (8)	Yes	Only cover of m4, 6, 16, 18, 20, 22 (wrap-around)
A'·C'·E'	m0, 2, 8, 10 (4)	Yes	Only cover of m8, m10 (wrap-around both ways)

(c) Minimal SOP: F = C·E + B'·E' + A'·C'·E' **Minimal POS:** F = (C + E')·(C + A' + B')·(E + B' + C')

(b) Algebraic cross-check: C·E covers all minterms with C=E=1; B'·E' covers all with B=E=0; A'·C'·E' adds the remaining terms (m8, m10). Each term has an essential minterm, and no literal can be dropped without covering a 0, so the form is minimal. D cancels out entirely (X·D + X·D' = X).

(d) Cost comparison:

Measure	Canonical SOP	Minimal SOP	Saving
Product terms	18 (5 literals each)	3 (2, 2, 3 literals)	83 % fewer
Total literals	90	7	92 % fewer
Gates	18 AND-5, 1 OR-18, 5 NOT	3 AND (2,2,3-in), 1 OR-3, 4 NOT	—
Variables used	A, B, C, D, E	A, B, C, E (D eliminated)	—

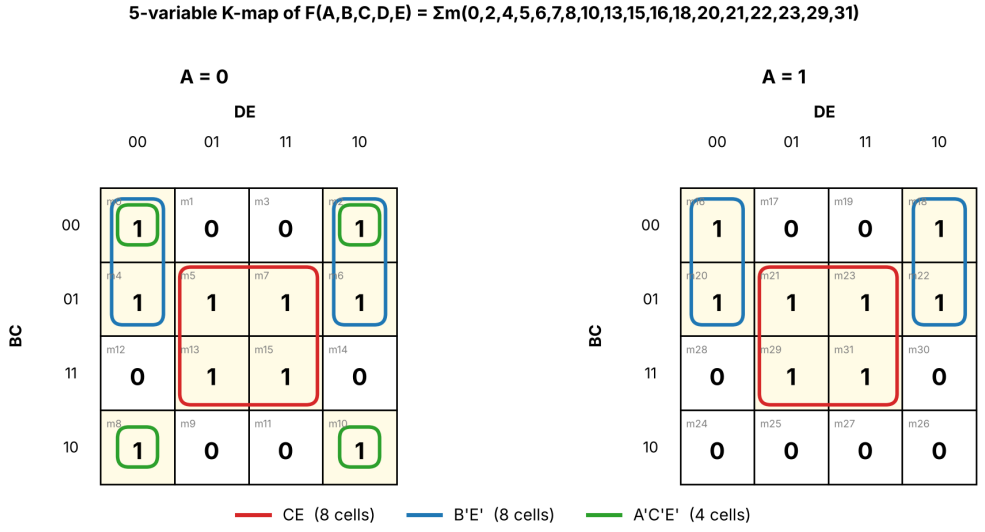


Figure 1. 5-variable K-map of F with the three groups (C·E red, B'·E' blue, A'·C'·E' green) in both A=0 and A=1 layers.

3. Logisim Implementation

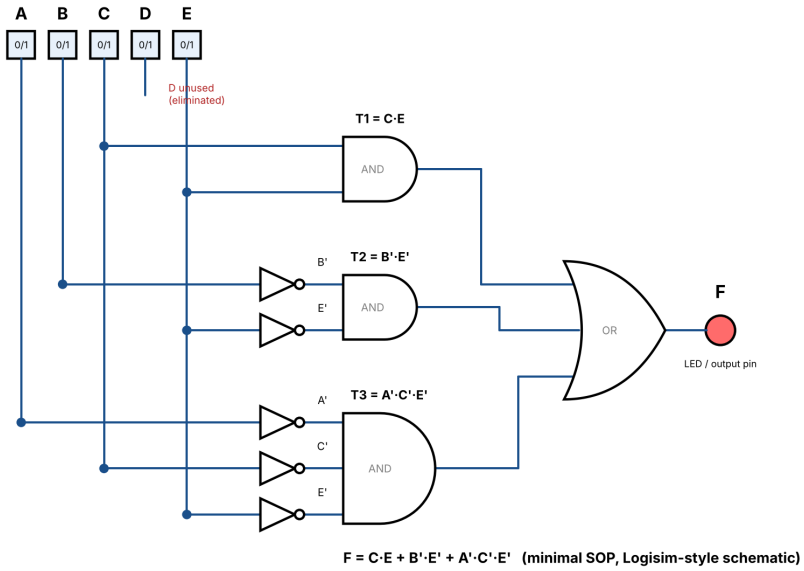


Figure 2. Logisim-style schematic of F = C·E + B'·E' + A'·C'·E' (D unused).

(e) Build steps: (1) Place five input pins named A–E and one output pin named F. (2) Add NOT gates for B and E (term 2) and for A, C and E (term 3); E' can be shared. (3) AND #1 (2 inputs): C, E. AND #2 (2 inputs): B', E'. AND #3 (3 inputs): A', C', E'. (4) OR gate with 3 inputs fed by the three AND outputs; OR output to pin F. (5) Leave D unconnected. (6) Verify with the Poke tool, or Project → Analyze Circuit (truth table and minimal expression).

4. Result

Verification: all 32 input combinations (Logisim output pin F vs original)

m	A	B	C	D	E	F (orig)	F (SOP)	F (POS)	Match
0	0	0	0	0	0	1	1	1	PASS
1	0	0	0	0	1	0	0	0	PASS
2	0	0	0	1	0	1	1	1	PASS
3	0	0	0	1	1	0	0	0	PASS
4	0	0	1	0	0	1	1	1	PASS
5	0	0	1	0	1	1	1	1	PASS
6	0	0	1	1	0	1	1	1	PASS
7	0	0	1	1	1	1	1	1	PASS
8	0	1	0	0	0	1	1	1	PASS
9	0	1	0	0	1	0	0	0	PASS
10	0	1	0	1	0	1	1	1	PASS
11	0	1	0	1	1	0	0	0	PASS
12	0	1	1	0	0	0	0	0	PASS
13	0	1	1	0	1	1	1	1	PASS
14	0	1	1	1	0	0	0	0	PASS
15	0	1	1	1	1	1	1	1	PASS

m	A	B	C	D	E	F (orig)	F (SOP)	F (POS)	Match
16	1	0	0	0	0	1	1	1	PASS
17	1	0	0	0	1	0	0	0	PASS
18	1	0	0	1	0	1	1	1	PASS
19	1	0	0	1	1	0	0	0	PASS
20	1	0	1	0	0	1	1	1	PASS
21	1	0	1	0	1	1	1	1	PASS
22	1	0	1	1	0	1	1	1	PASS
23	1	0	1	1	1	1	1	1	PASS
24	1	1	0	0	0	0	0	0	PASS
25	1	1	0	0	1	0	0	0	PASS
26	1	1	0	1	0	0	0	0	PASS
27	1	1	0	1	1	0	0	0	PASS
28	1	1	1	0	0	0	0	0	PASS
29	1	1	1	0	1	1	1	1	PASS
30	1	1	1	1	0	0	0	0	PASS
31	1	1	1	1	1	1	1	1	PASS

Figure 3. Verification of all 32 input combinations: Logisim output F versus the original, SOP and POS forms (all PASS).

Observation: for all 32 combinations the original canonical form, the SOP circuit and the POS form give the same output. F = 1 for exactly the 18 listed minterms, so the simplification is correct (32/32 rows match). Literals drop from 90 to 7 (92 % fewer) and AND gates from 18 to 3 (83 % fewer); the largest OR fan-in drops from 18 to 3.

5. Conclusion

GenAI reduced F(A,B,C,D,E) to the minimal SOP CE + B'E' + A'C'E' and the equivalent POS (C + E')(C + A' + B')(E + B' + C'), identifying three essential prime implicants and eliminating variable D. The circuit uses 3 AND gates, 1 OR gate and 4 NOT gates and matched the original function on all 32 inputs. GenAI speeds up Boolean minimization, but its output must still be verified independently (K-map, algebra, truth table and simulator), because AI answers can contain errors. This workflow gives a reliable design with lower hardware cost, propagation delay and power.

AI-Based Fault Detection in a Digital Full Adder

Design of a deliberately faulty circuit, GenAI diagnosis from its truth table, correction and re-verification

1. Prompt

You are a digital logic test engineer. A three-input block (inputs A, B and carry-in C) is supposed to perform binary addition, producing a Sum bit and a carry-out bit. Its observed behaviour is given below as a truth table. The Sum column agrees with the specification, but one carry-out entry does not. (1) Compare every row with the specification and name the exact row and output that fail. (2) Work out which product term the faulty carry-out expression is missing. (3) Propose a corrected sum-of-products carry-out expression and re-check all eight rows against the specification. Present the rows in a table, then explain your reasoning in one short paragraph.

Observed table (A, B, C | Sum | Cout): 000|0|0 001|1|0 010|1|0 011|0|1 100|1|0 101|0|**0** 110|0|1 111|1|1

2. Prompt Response (GenAI output)

Row check: the Sum column matches the specification in all 8 rows, so the XOR chain is healthy. The carry-out column fails in one row only: A=1, B=0, C=1, where the specification requires Cout=1 but the block gives 0.
Diagnosis: the observed carry-out behaves like AB + BC. Both terms are 0 when A=1, B=0, C=1, so the output is 0. The product term **AC** is missing from the carry-out logic. This is a single missing-term fault.
Correction: Cout = AB + BC + AC, which is equivalent to the majority function of A, B and C.
Re-verification: with the corrected expression, all 8 rows match the specification for both Sum and Cout.

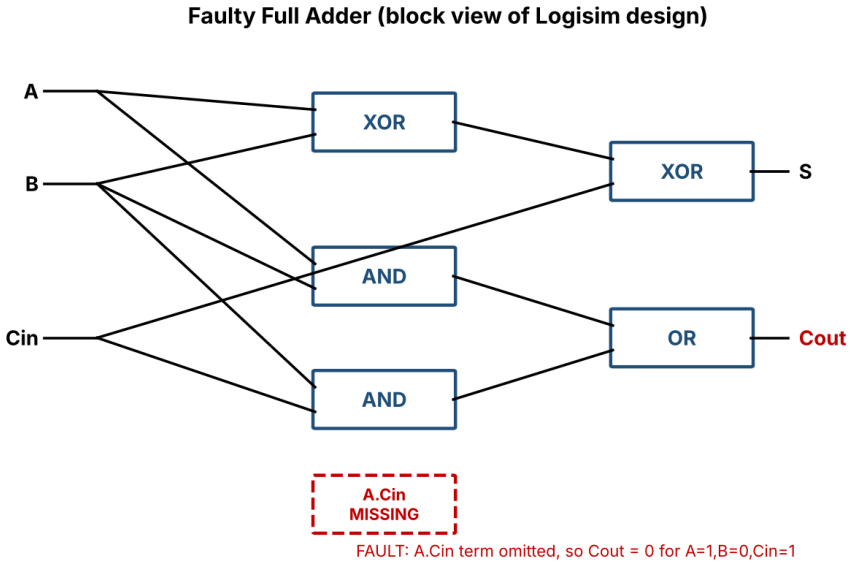


Figure 1. Faulty full adder. The dashed box is the missing A.Cin product term.

3. Logisim Implementation

Faulty build: create a project and a circuit named FullAdder_Faulty. Add three 1-bit input pins (A, B, Cin) and two output pins (S, Cout). From the Gates library place two 2-input XOR gates, two 2-input AND gates and one 2-input OR gate. Wire A and B into the first XOR, the XOR output and Cin into the second XOR, and the second XOR to S. Wire A·B and B·Cin into the two AND gates, then both AND outputs into the OR gate, and the OR to Cout. Do not place a third AND gate for A·Cin; this omission is the injected fault. Run Project → Analyze Circuit to generate the truth table and compare it with the specification.
Correction: save a copy as FullAdder_Corrected. Add a third AND gate with inputs A and Cin, change the OR gate to 3 inputs, and connect the three AND outputs to it. Run Analyze Circuit again and confirm that the table matches the specification.

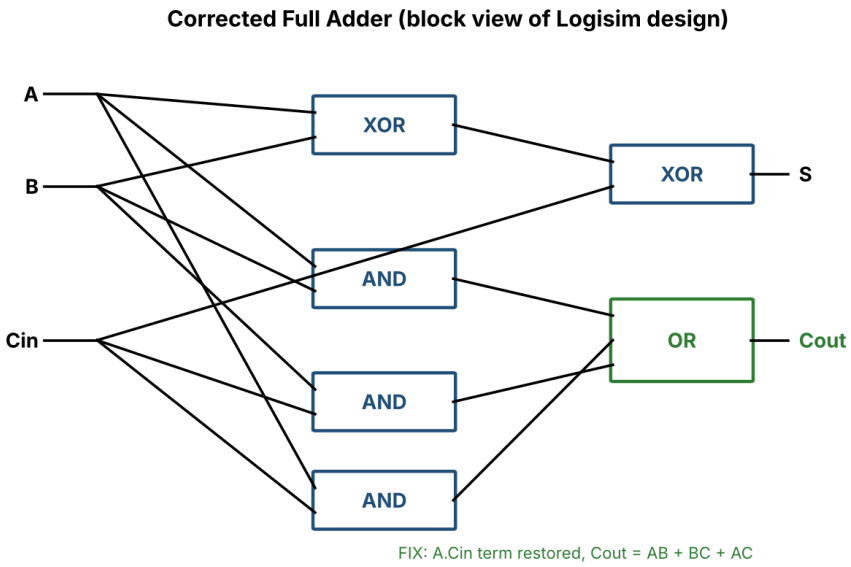


Figure 2. Corrected full adder with the A·Cin term restored (green).

4. Result

A	B	Cin	Sum (spec)	Cout (spec)	Cout (faulty)	Cout (corrected)	Match
0	0	0	0	0	0	0	Pass
0	0	1	1	0	0	0	Pass
0	1	0	1	0	0	0	Pass
0	1	1	0	1	1	1	Pass
1	0	0	1	0	0	0	Pass
1	0	1	0	1	**0**	1	Pass
1	1	0	0	1	1	1	Pass
1	1	1	1	1	1	1	Pass

Before correction, one of eight carry-out rows fails (A=1, B=0, Cin=1, shown in red). After correction, all eight rows pass for Sum and Cout. The Sum output was correct in the faulty design and is unchanged by the fix.

5. Conclusion

The prompt-based diagnosis located the faulty row and the missing A·Cin product term from the truth table alone, without access to the schematic. Applying the correction and re-running the truth-table analysis confirmed the fix, so the workflow of design, fault injection, AI diagnosis, correction and re-verification works for a small combinational circuit. The method scales to larger blocks, where exhaustive manual checking becomes impractical. Its limitation is that the AI's diagnosis is only as reliable as the truth table it receives, so simulation remains the final check.

Automatic Parking-Barrier Controller – Combinational + Sequential Design (CO2)

Moore FSM, 2 D flip-flops, simulated in Logisim / CircuitVerse

1. Prompt (given to GenAI)

> "Act as a digital-logic design tutor. I am doing a lab applying **combinational and sequential design** to a **real automation scenario**; I choose an **automatic parking-barrier controller**. (1) **Define inputs, outputs and constraints**. (2) **Derive the FSM / state table** with state encoding, then the **combinational logic** (next-state and output equations, K-map simplified) using D flip-flops, and check my state table. (3) Explain how to **implement it in Logisim/CircuitVerse** (components, wiring, clock). (4) Give a **test plan against all input scenarios** (every state × every input combination) and **document the edge cases**. Keep it concise, in tables."

2. Prompt Response

(1) Inputs, outputs, constraints

Signal	Type	Meaning
A	Input	Vehicle detected at the entry loop (1 = present)
V	Input	Valid ticket / payment accepted (1 = valid)
X	Input	Safety loop under the barrier (1 = vehicle under/at barrier)
CLK	Input	Single synchronous clock (one tick = one control step)
UP / DOWN	Output	Motor raises / lowers the arm (must never be 1 together)
OPEN / CLOSED	Output	Green / red signal lamps

Constraints: synchronous, Moore outputs (depend only on state); barrier opens only if **A·V = 1**; barrier must **never close while X = 1** (safety); if X appears while closing it must **re-open**; power-up state = CLOSED (00); all 4 states of 2 bits are used, so no unused/lock-up state; opening/closing is assumed to take one clock step.

(2) FSM state table (Gray-code assignment Q1Q0) and combinational logic

State	Q1Q0	Meaning	Next state	Outputs (UP, DOWN, OPEN, CLOSED)
S0	00	Closed, idle	A·V ? S1 : S0	0 0 0 1
S1	01	Opening	S2 (always)	1 0 0 0
S2	11	Open, car passing	X ? S2 : S3	0 0 1 0
S3	10	Closing	X ? S1 (re-open) : S0	0 1 0 0

K-map result (D flip-flop: D = next Q): $D_1 = Q_0$ | $D_0 = Q_1'\cdot(Q_0 + A\cdot V) + Q_1\cdot X$
Outputs: CLOSED = $Q_1'Q_0'$, UP = $Q_1'Q_0$, OPEN = Q_1Q_0, DOWN = Q_1Q_0'
Derivation: in S0 (00) D_0=A·V; in S1 (01) D_0=1; in S2 (11) and S3 (10) D_0=X, so $D_0 = Q_1'Q_0'AV + Q_1'Q_0 + Q_1X = Q_1'(Q_0+AV) + Q_1X$. D_1 is 1 only for Q_1Q_0 = 01 or 11, i.e. $D_1 = Q_0$.

(3) Logisim build steps: place 3 input pins (A, V, X) + clock; AND(A,V) → OR with Q0 → AND with Q1' → OR with AND(Q1,X) gives D0; wire Q0 straight to D1; two D flip-flops (clock = same CLK, Q1' / Q0' from the flip-flop Q' pins); four 2-input AND gates decode the state to LEDs (CLOSED, UP, OPEN, DOWN). **(4) Test plan:** exhaustive 4 states × 8 inputs = 32 cases plus a sequence test (Section 4).

3. Logisim Implementation

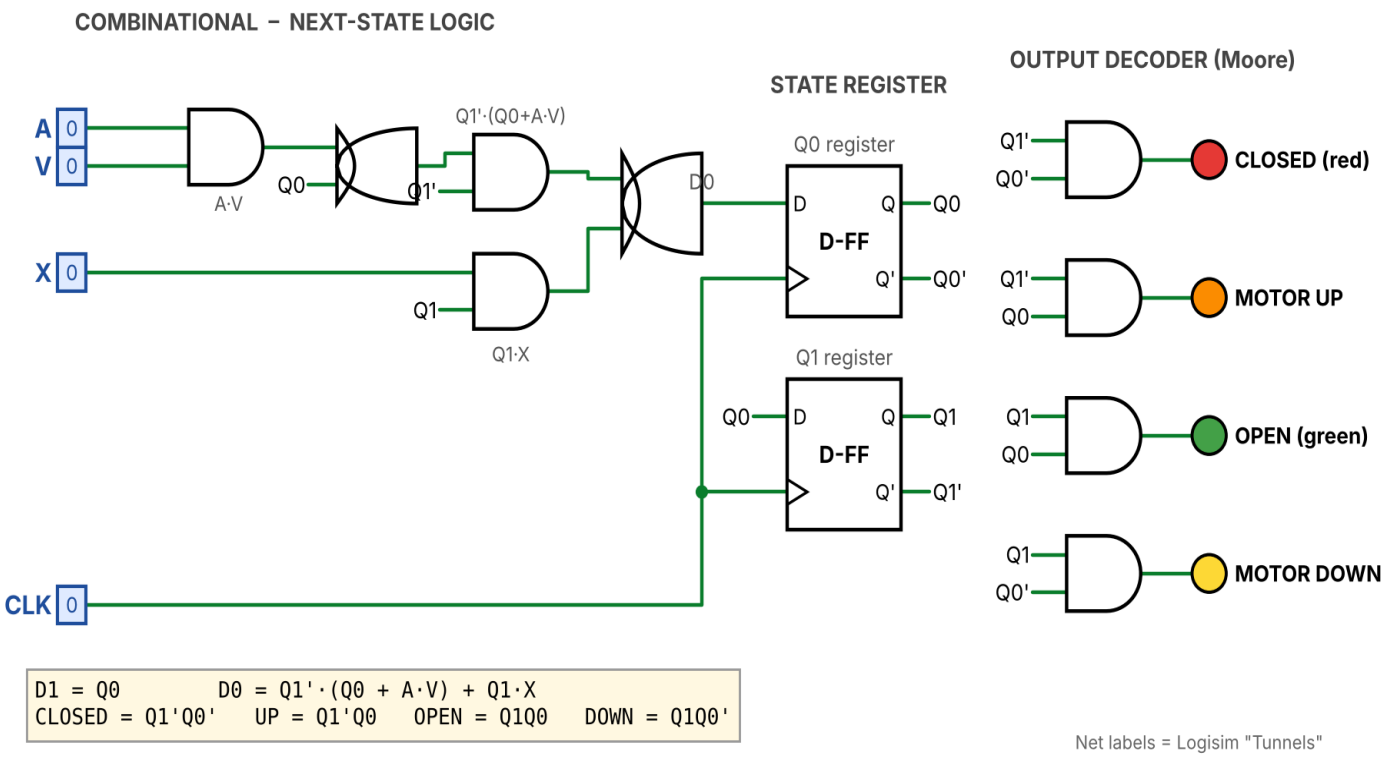


Fig. 1 – Logisim/CircuitVerse circuit of the controller (net labels are tunnels). Replace/append your own screenshot from the simulator if required by your lab.

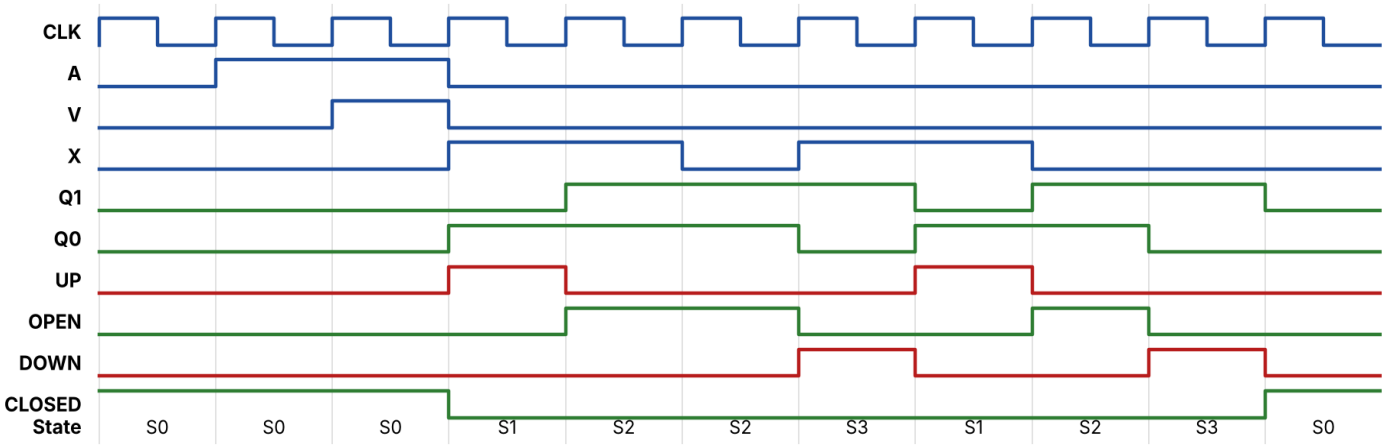


Fig. 2 – Timing/simulation waveform for the test sequence: no ticket (stays closed) → open → car passing (X=1, stays open) → closing → car re-enters (re-open) → closed.

4. Result

Exhaustive test – next state (and outputs of the present state) for every state and input combination A V X. The Boolean equations were checked against the state table: **32 / 32 cases match**.

Present state	AVX=000	AVX=001	AVX=010	AVX=011	AVX=100	AVX=101	AVX=110	AVX=111	UP DN OPEN CLSD
S0 (00)	S0 (00)	S0 (00)	S0 (00)	S0 (00)	S0 (00)	S0 (00)	S1 (01)	S1 (01)	0 0 0 1
S1 (01)	S2 (11)	S2 (11)	S2 (11)	S2 (11)	S2 (11)	S2 (11)	S2 (11)	S2 (11)	1 0 0 0
S2 (11)	S3 (10)	S2 (11)	S3 (10)	S2 (11)	S3 (10)	S2 (11)	S3 (10)	S2 (11)	0 0 1 0
S3 (10)	S0 (00)	S1 (01)	S0 (00)	S1 (01)	S0 (00)	S1 (01)	S0 (00)	S1 (01)	0 1 0 0

Edge cases documented

#	Scenario	Observed behaviour
1	A=1, V=0 in S0 (no valid ticket)	Stays S0, barrier closed
2	V=1, A=0 or X=1 only, in S0	Ignored; stays closed (no false opening)
3	X held at 1 in S2	Stays S2 (open) indefinitely – car never hit
4	X=1 while closing (S3)	S3→S1→S2: arm reverses and re-opens (safety)
5	A·V or X changes in S1	Ignored; S1 always goes to S2 (no retrigger / glitch)
6	A·V=1 during S3 (next car right behind)	Not accepted in S3; after S0 is reached the next tick opens it (1-cycle delay)
7	Power-up / reset	Flip-flops start at 00 → CLOSED, UP=DOWN=0
8	UP and DOWN together	Impossible: they decode different states (01 vs 10)

Sequence test (Fig. 2): state trace S0 → S0 → S0 → S1 → S2 → S2 → S3 → S1 → S2 → S3 → S0 for the input sequence AVX = 000, 100, 110, 001, 001, 000, 001, 001, 000, 000 – exactly as expected.

5. Conclusion

A real automation task (parking barrier) was modelled as a 4-state Moore FSM. The combinational logic ($D_1 = Q_0$, $D_0 = Q_1'(Q_0+AV)+Q_1X$ and four AND output decoders) together with two D flip-flops realises the sequential behaviour with only 6 gates plus 2 flip-flops. All 32 state/input combinations and the safety edge cases (no ticket, car under barrier, re-open while closing, power-up) behave as specified, so the objective of applying combinational and sequential design (CO2) is met. GenAI helped derive and cross-check the state table, but the equations were verified independently by exhaustive comparison.

Digital Clock FSM Design with GenAI (24-Hour, Flip-Flop Based)

1. Prompt

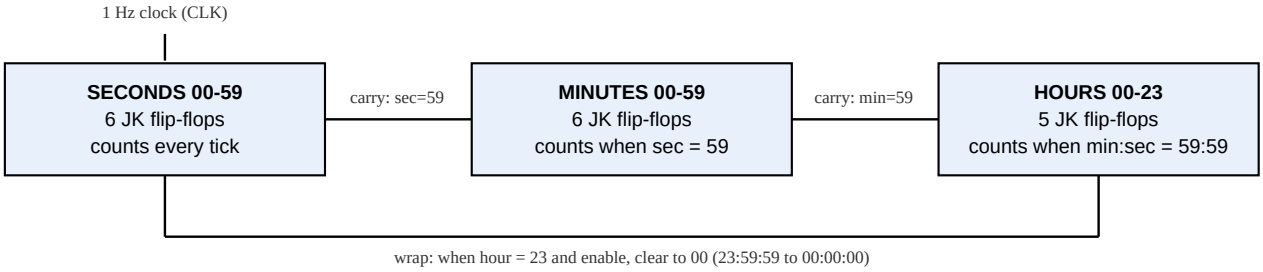
Act as a digital logic designer. Using a generative AI assistant, design a 24-hour digital clock as a finite state machine. Derive the state set for seconds, minutes and hours, the state encoding, and the next-state rules. Then convert those rules into a flip-flop implementation with the carry chain between stages and the hour wrap from 23 back to 00. Give the excitation equations, a Logisim build plan, and a verification plan that counts seconds, minutes and hours through a full day, including the rollover points 59 to 00, 59:59 to 00:00 and 23:59:59 to 00:00:00.

2. Prompt response (GenAI output, condensed)

The assistant treated the clock as one FSM whose state is the triple (HH, MM, SS), giving 24 x 60 x 60 = 86,400 states. It proposed a factored design: three counters with binary encoding, where each counter advances only when the lower-order counter rolls over. Seconds use 6 flip-flops, minutes use 6, and hours use 5, for 17 flip-flops in total. Each flip-flop is a JK type with J = K = T, so it toggles when T = 1 and holds when T = 0. The response derived the toggle conditions as AND chains: the seconds bit i toggles when all lower seconds bits are 1; the minutes stage is enabled when seconds are 59; the hours stage is enabled when minutes and seconds are both 59. The hour counter is cleared synchronously when it reaches 23, which implements the 24-hour wrap. The response also suggested a simulation check against a reference time value, which is used in Section 4.

Counter	Range	Bits (FFs)	Toggle (T) equation for bit i	Enable from lower stage
Seconds	00 to 59	6	T0 = 1; Ti = AND(s0..s(i-1))	sec_roll = (sec == 59)
Minutes	00 to 59	6	T0 = sec_roll; Ti = AND(sec_roll, m0..m(i-1))	min_roll = sec_roll AND (min == 59)
Hours	00 to 23	5	T0 = min_roll; Ti = AND(min_roll, h0..h(i-1)); clear when h = 23	wrap = min_roll AND h = 23

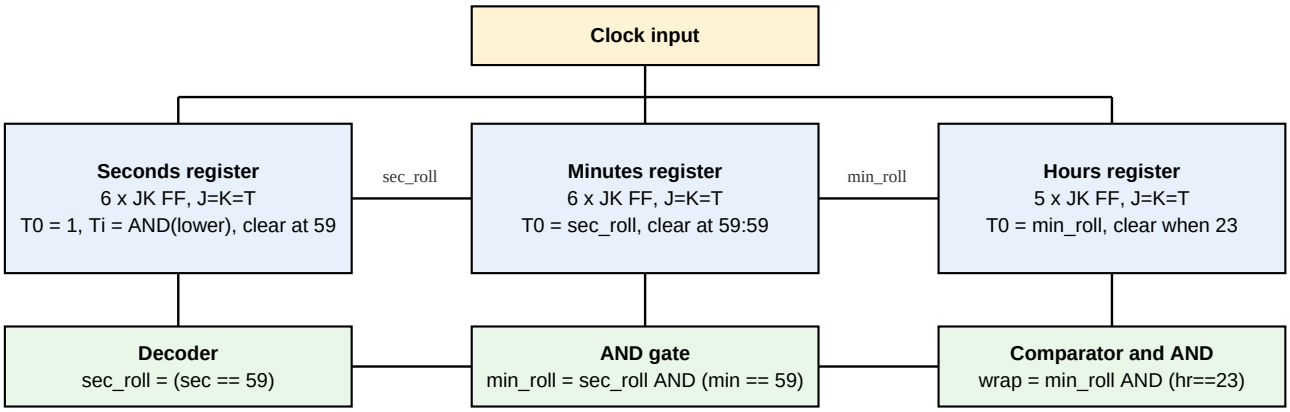
Figure 1. State and carry chain of the clock FSM (generated solution diagram)



3. Logisim Implementation

The circuit uses 17 JK flip-flops driven by one 1 Hz clock (in Logisim, a Clock component with a tick frequency set to 1 Hz for demonstration, or a faster tick for quicker testing). Build it in three blocks. First, place the seconds register: six JK flip-flops with J and K tied to a shared toggle line, bit 0 tied to constant 1, each higher bit's toggle formed by an AND gate of the lower Q outputs, and a clear driven by a sec == 59 decoder. Second, place the minutes register: the same structure, with bit 0's toggle equal to sec_roll from a seconds decoder (sec == 59) and a clear driven by min_roll. Third, place the hours register: five flip-flops, with toggles gated by min_roll, and a clear line that is driven by an AND of min_roll and a 5-bit comparator equal to 23 (10111). Connect the clear to the asynchronous or synchronous reset of the hour flip-flops as shown in Figure 2, and display the seconds, minutes and hours outputs with Logisim hex or binary display components.

Figure 2. Logisim block schematic of the cascaded counters (generated solution diagram)



4. Result

A bit-level simulation of the same JK flip-flop netlist was run for one full day, 86,400 clock ticks, starting from 00:00:00. After each tick, the register contents were decoded into HH:MM:SS and compared with a reference time that advances by one second. The circuit produced 0 mismatches. Counting events matched the expected values, as shown below.

Check	Expected	Measured
Clock ticks simulated	86400	86400
Mismatches against reference time	0	0
Seconds rollovers (59 to 00, minute increments)	1440	1440
Minutes rollovers (59:59 to 00:00, hour increments)	24	24
Day wraps (23:59:59 to 00:00:00)	1	1
After 59 ticks	00:00:59	00:00:59
After 60 ticks	00:01:00	00:01:00
After 3599 ticks	00:59:59	00:59:59
After 3600 ticks	01:00:00	01:00:00
After 86399 ticks	23:59:59	23:59:59
After 86400 ticks (end of day)	00:00:00	00:00:00

5. Conclusion

The GenAI prompt turned a single design goal into a clear FSM: 86,400 states, factored into seconds, minutes and hours counters with carry enables and an hour wrap at 23. Mapping that into 17 JK flip-flops with J = K = T gave a compact implementation whose toggle equations come straight from the AND chains. The simulation shows that the seconds, minutes and hours counters increment correctly, roll over at 59, 59:59 and 23:59:59, and return to 00:00:00 after exactly one day. Note that the verification above is a flip-flop-level simulation of the same equations; the Logisim schematic in Figure 2 should be built from the Section 3 steps and checked with the same expected values in the Result table.